Towards a Smart Opponent for Board Games: Learning beyond Simulations

Naveen Karunanayake*[1], Achintha Wijesinghe*[1], Chameera Wijethunga*[1], Chinthani Kumaradasa[1], Peshala Jayasekara[1], Ranga Rodrigo[1]

naveenharshitha95@gmail.com, *ac9ya5@gmail.com*, *chameeracjw@gmail.com*, *ckumaradasa@gmail.com*, *peshala@uom.lk*, *ranga@uom.lk*

***Abstract*—Reinforcement learning algorithms have been successfully trained for games like GO, Atari, and Chess in simulated environments. However, in cue sport-based games like Carrom, real world is unpredictable unlike in Chess and GO due to the stochastic nature of the gameplay as well as the effect of external factors such as friction combined with multiple collisions. Hence, solely training in a simulated platform for games like Billiard and Carrom, which need precise execution of a shot, would not be ideal in actual gameplay. This paper presents a real-time vision based efficient robotic system to play Carrom against a proficient human opponent. We demonstrate the challenges of adopting a reinforcement learning algorithm beyond simulations in implementing a strategic gameplay for the robotic system. We currently achieve an overall shot accuracy of 70.6% by combining heuristic and reinforcement learning algorithms. Analysis of the overall results suggests the possibility of adopting a real-world training for board games which need precise mechanical actuation beyond simulations.**

***Index Terms*—Robotic systems, Machine vision, Image processing, Machine learning, Heuristic algorithms**

I. Introduction

Robotic systems are widely used in a diverse range of fields such as Healthcare, Agriculture, Manufacturing, and Military. Besides, researchers have shown interest in developing robots for games like Pool [1], Chess [2], [3], Table Tennis [4], [5], Jenga, etc. Initially, brute force algorithms were used in the gameplay, but with the development of artificial intelligence and machine learning, different learning techniques have been successfully applied to build agents that can master the skills of games like Chess, Atari [6] and GO [7]. GO, in particular, uses reinforcement learning algorithms such as AlphaGO [8] and AlphaZeroGO [9].

For games like Chess and GO, simulated platforms can be used to train algorithms and then used in robotic systems as the gameplay in real world does not differ from a simulated environment. On the other hand, training an algorithm in a simulated environment for cue sport-based tabletop games like Billiard, Carrom, and Novuss will not match with the real game as the stochastic behaviour of the game cannot be predicted precisely.

Carrom is a popular tabletop game mainly in Asia and Europe, which is also referred to as finger billiards. It is known to be the ancestor of Pool, Snooker, and Billiards. It can be played as a two or four player (two teams) game. Players are assigned a colour (white or black), and the first to pocket all the pieces including, the queen, wins the game. Here, we implement a smart Carrom playing robotic system with four degrees of freedom. Our robot implementation is shown in Figure 1. Further, in order to implement a strategic play, we integrate the knowledge of reinforcement learning concepts.

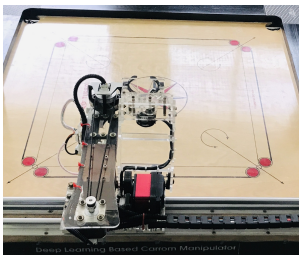


Fig. 1: System Implementation

Reinforcement learning techniques [10] have evolved significantly since the introduction of traditional Q learning [11] algorithms. From expert level gaming agents to self driving cars, reinforcement learning concepts have conquered a wide range of fields. One major challenge in reinforcement learning algorithms is the trade-off between exploration and exploitation [12]. It becomes more challenging when adopting these algorithms for training robotic systems beyond simulations. Reinforcement learning algorithms with zero prior knowledge require a large amount of training time and data for complex environments. In simulated environments, it is possible to recreate any state repeatedly unlike, in real-world scenarios. Hence, training such an algorithm on a robotic system is an arduous task. We show the effectiveness of employing a heuristic algorithm in speeding up the training process of the learning model.

In this paper, we present our design philosophy and the system implementation along with the results of our smart Carrom opponent. We leverage the knowledge of computer vision and robotics to automate the gameplay. To the best of our knowledge, our work is the first to attempt a reinforcement learning training approach in a real world implementation of a cue sport-based tabletop game where the continuity of both state and action spaces, and the stochastic behavior exacerbate

*These authors contributed equally

[1]Department of Electronic and Telecommunication Engineering, University of Moratuwa, Sri lanka

the game play. The remainder of the paper is organised as follows. In Section II, we present the methodology we adopted. Section III presents our results, while Section IV discusses implications of our findings, limitations and possible future improvements. Section V concludes the paper.

II. Method

The main feature of this paper is the smart Carrom playing robotic system. Further we try to adopt this system to train a reinforcement learning model. For convenience, we break down the overall system into four major technical blocks. In achieving an end-to-end Carrom manipulator, firstly, we use a machine vision based algorithm to capture the board configuration and map it into the rectangular coordinate system by defining a reference point. A camera mounted on top of the Carrom board captures an image of the board and feeds it to the piece detection algorithm for further processing. The algorithm outputs an array of size (23x2), which is referred as the coordinate matrix. Rectangular coordinates of nine white pieces, red piece, nine black pieces, and four pockets are stacked together in this matrix. For the reinforcement learning block, the coordinate matrix plays the role of the state space. We input the coordinate matrix to both rule-based and reinforcement learning blocks. Both algorithms output shot parameters in terms of disk position d (disk location), angle θ (angle the manipulator needs to be rotated), and velocity v (shooting speed given to the manipulator). Shots are selected from both reinforcement learning and rule-based algorithms according to a given probability. Carrom robot manipulator, which is the final block of the system, executes the shot according to given parameters (d, θ, v). Figure 2 shows the overall architecture and the interconnection of the major blocks. Each technical block is discussed further in the following subsections.

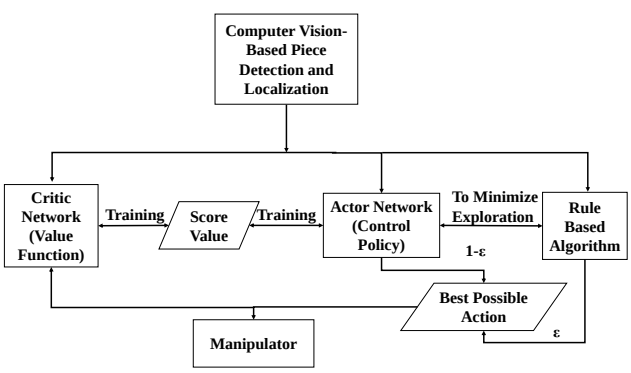


Fig. 2: System Architecture

A. Machine Vision Based Piece Detection and Localization

Machine vision based piece detection and localization block acts as the eyes of the system. It identifies the pieces and pockets and maps the board to a rectangular coordinate system. A camera is mounted on top of the Carrom board to capture images during gameplay. The captured image is processed, using Algorithm 1 to determine the coordinate matrix. The system is implemented using Python and it provides the required real-world expectations. The mapping of the pixel domain to the rectangular coordinate system, gives an advantage in terms of computational complexity and efficiency. Figure 3 represents the output of the piece detection algorithm, applied to a captured board configuration.

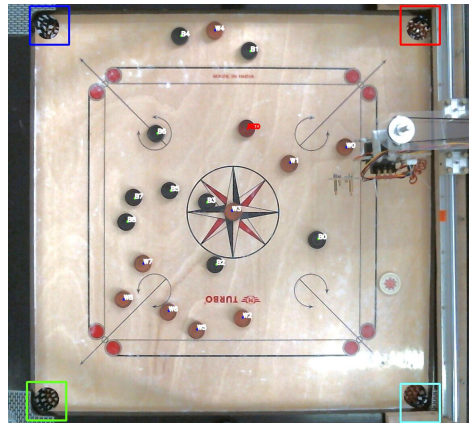


Fig. 3: The output of the Detection Algorithm 1: The figure illustrates the capability of detecting each red, white, and black piece. The detection of the white pieces is obscured due to the similarity in the board's and white pieces' texture.

B. Rule-Based Algorithm

The output location matrix of the pieces and pockets from piece detection and localization block is fed to the rule-based algorithm. We use geometry to filter out shots with minimum interference, and conservation of momentum and energy to calculate the shooting speed and the angle of the robot manipulator. The main purpose of this heuristic algorithm is to leverage it to minimize the initial exploration done in the training process of the reinforcement learning algorithm. We present the rule-based algorithm from one player's perspective (black) in Algorithm 2 in the Appendix.

Furthermore, the friction of the board is another critical factor that needs to be taken into account during gameplay. To approximately counter the effect of friction, we fit a polynomial function to map the calculated shooting speed (pixels per second) from the rule-based algorithm to the manipulator servo input. The degree of the polynomial is selected as 2, after doing an experiment with the degree of the polynomial as the hyper-parameter on a collected dataset. We present the graph of the selected case in Section III.

C. Robot Manipulator and Hardware Architecture

In this section, we present the manipulator hardware architecture with four essential mechanisms, the linear actuator, the rotator, the shooter, and the disk placer. Figure 4 illustrates these mechanisms, while the following subsections explain each in detail.

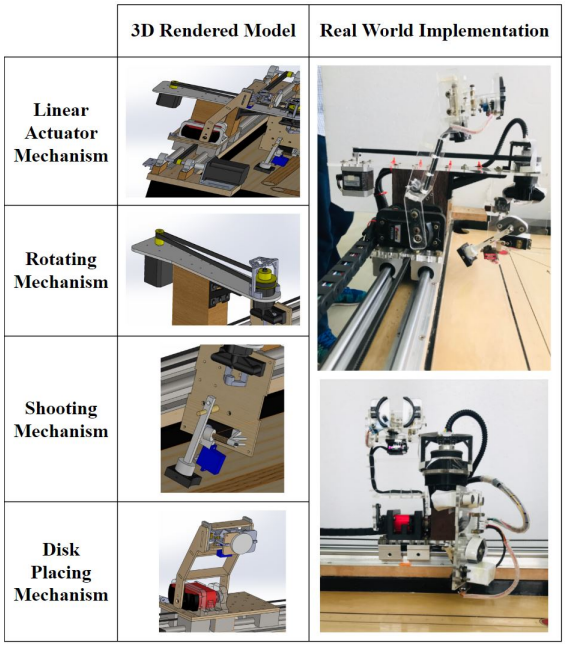


Fig. 4: Illustration of Proposed Manipulator (3D Rendered) Design and the Real-World Implementation.

1) Linear Actuator Mechanism: The fundamental requirement of the linear actuator mechanism is to move the shooter and the disk placer along the lateral axis of the robot, providing the ability to either place or shoot the disk accordingly. For a prismatic motion, the most common alternatives are either the lead/ball-screw mechanism or the belt-pulley mechanism. According to the calculations shown in Figure 14 in Appendix, the required torque and the speed can be achieved using a belt pulley mechanism. For the implementation, we use a stepper motor and an open-loop teeth-belt. The choice of a stepper motor outsmarts other alternatives due to open-loop, accuracy, simple controllability, and availability of micro-steps. Service free high-end linear railings ensure robust and smooth motion while two limit switches placed at the extremes of the moving platform take care of calibration and handling abnormalities.

2) Rotating Mechanism: The belt pulley mechanism is the primary mechanism used in implementing the rotating mechanism. It rotates the shooter around the disk providing the ability to play 360° shots. In addition, it produces lesser spatial obstruction to the camera to capture the board configurations since the camera is mounted on top of the board. The rotating mechanism is capable of turning ± 180° so the shooter can execute shots along any direction on the board surface. Low gear makes smoother and higher resolution rotations of the shooter, as it hangs by two ball bearings. Hall effect based absolute encoder comes to play in measuring the angle rotated to take feedback and to execute the initial localization process.

3) Shooting Mechanism: From the natural finger motions of a Carrom player, we only need the shooting mechanism to vary the momentum given to the disk which allows to control the initial velocity of the disk. Therefore, we use a gravity-based shooting mechanism, which provides the required kinetic energy to the disk, enabling it to move along the line of collision. A servo motor lifts the shooting-lever to the desired height, which is calculated by back-tracing the initial velocity needed to be given to the disk.

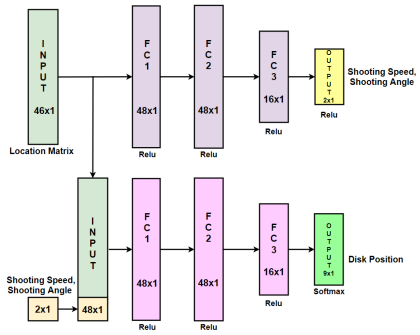
4) Disk Placing Mechanism: The disk placing mechanism which places the disk on the allowed area of the board consists of a two- degrees-of-freedom manipulator. Two servo motors take care of the actuation and a ball bearing bears the weight of the disk lowering arm. Spur gear based parallel grabber grabs the disk tightly and releases in an upward motion without disturbing nearby pieces. An infra-red reflectivity-based proximity sensor senses the placing of the disk and the coupled micro-controller interrupt handles passing of the signal to the PC. The robot gets the chance to play whenever the disk is placed by the human opponent on the disk-placer.

D. Reinforcement Learning Approach

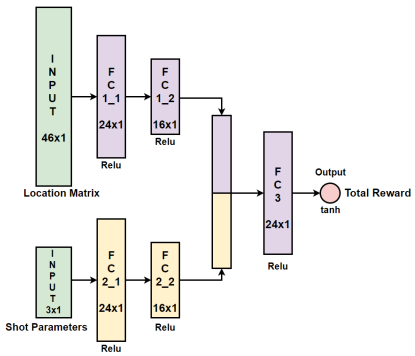
Heuristic algorithms alone would not be sufficient to outsmart human skill levels as there are infinite number of scenarios and possible actions that we may come across during gameplay, which makes it impractical to hand code all these cases manually. Because of that, we need to employ a method that learns these different scenarios incrementally with its own experience. Further, winning a game would need a strategic way of playing. Pocketing the easiest pieces first might not be the best option in winning a game. A player should try to pocket his pieces while blocking opponents pieces. These types of strategies can only be learnt through experience. Reinforcement learning techniques have proven to be efficient in solving problems of that kind. Mapping basic terminology into our problem is as follows. The "state" is the configuration of the board at a given instance, "agent" is the Carrom playing manipulator, "environment" is the Carrom board, and "action" is the execution of a shot according to given parameters. Finally, "reward" is the feedback received by the manipulator based on the performance of the shot, which is evaluated by equation (1) considering the change in positions of the pieces before and after executing a shot.

The complexity of games like Carrom increases due to its continuous state and action spaces [13]. To address this inherent complexity of the game, we use the "actor-critic" approach [14], which is the fundamental approach for continuous state and action spaces scenarios. In the game of Carrom, winning strategy of a game involves choosing the best possible action at a given instance and also the effect of the choice for the overall game. In reinforcement learning, value-based methods are in favor of finding the best action for a given state. On the other hand, policy-based methods are used in episodic tasks. A hybrid version of both of these methods compensates for our requirement. This hybrid method is the actor-critic approach. We use the actor-critic model in our implementation to have a better score function. Using TD learning [15], we update the policy at each step. Traditional policy update would follow a Monte Carlo [16] approach where the policy update is done at the end of the episode using the total reward $R(t)$. Here, we implement a critic model that can approximate the

value function and use it to replace the reward function in the policy gradient.



(a) Architecture of the Actor



(b) Architecture of the Critic

Fig. 5: Actor-Critic Model Architectures: Both architectures implemented using fully connected (FC) layers. Choice of the activation functions are ReLU, tanh and softmax.

Figure 5 portrays the architectures for actor and critic models. The actor model takes the coordinate matrix, a vector of size 46 where the (x, y) pixel coordinates of the coordinate matrix are stacked together. The first pipeline takes, above stacked co-ordinate matrix as the input and predicts the shooting speed and shooting angle. We customize the input to the second pipeline by stacking the shooting speed and the shooting angle to the coordinate matrix. This outputs the position of the board where we should place the disk to play the given shot. On the other hand, the critic model has two parallel fully-connected layers, but with different sizes of input vectors. We use the actor to predict the shot parameters for a given instance in the game and the critic to use these shot parameters and the given instance to predict the total maximum future reward that can be gained by the actor. Using this feedback of the critic, we train the actor to predict the best possible shot parameters for a given instance.

1) Reward Function: We formulate the reward function considering whether the executed shot resulted in pocketing an opponent's piece, own piece, the disk or how likely the next shot will result in pocketing a piece. Thus, we fix four radial basis functions on the four pockets as shown in Figure 6.

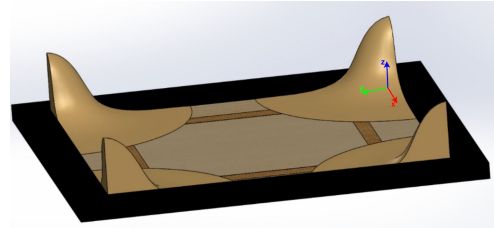


Fig. 6: 3D View of the reward function (1)

$$R = r(T)_{n_W} - r(T)_{n_B} + r(T-1)_{n_B} - r(T-1)_{n_W} \quad (1)$$

Where;

$r(t)_{n_k} = \sum_{i=1}^{n_k} \sum_{j=1}^{4} \exp\left(-(||h_j - p_i||d_j)^2\right)_{t=t}$

d_j - $||h_j - c||$

n_W - Number of white pieces ; n_B - Number of black pieces

h_j - Coordinates of hole j ; p_i - Coordinates of piece i

c - Coordinates of the center of the board

Also, pocketing an opponent's piece or disk will be given a reward of -1.

III. Results

In this section, we present the performance of each technical block and the overall system.

A. Machine Vision based Piece Detection and Localization

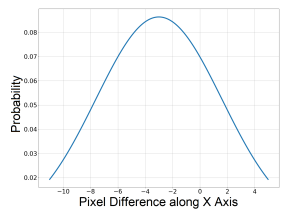
A prime factor for the success of the overall system is the performance of the piece detection and localization algorithm as incorrect center coordinates of pieces and holes will lead to incorrect shot parameters. In Table I, we tabulate the variation of the detection rates with different intensity ranges. We observe 100 instances of pieces of each color for each intensity range to calculate the detection rates. The rate of detecting black pieces is very high and almost stable. In contrast, detection rate of white pieces drops significantly for several intensity levels due to the subtle difference between the color of the board surface and white pieces. Also, the patterns that come with the board affect adversely for the detection algorithm. Figure 7, shows the center pixel co-ordinate deviation from the true center pixel co-ordinate of white and black pieces. It illustrates the probability distribution of the pixel deviation of the center coordinates, that can be expected in a given game play. Empirical results show that the center value deviation is under an acceptable range. From these results, we choose the most favorable conditions to set up the environment.

B. Rule-Based Algorithm

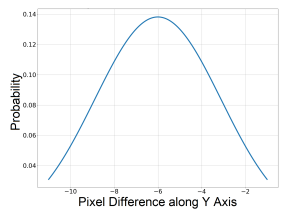
The rule-based algorithm plays a central role in reducing exploration time. In the robotic system, execution of a shot depends on the parameters of the shot. A random combination of each parameter (d, θ, v) without any prior knowledge is highly unlikely to collide with a piece, as each of these

TABLE I: White and Black Piece Detection Rate Against the Luminous Value.

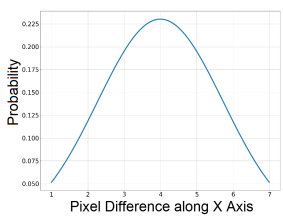
Luminous value range (lx)	**White Pieces Detection Rate (%)**	**Black Pieces Detection Rate (%)**
1200-1400	100	100
900-1000	83.33	100
600-700	82.5	99
400-500	88.8	98
200-300	87.5	100
50-100	100	100



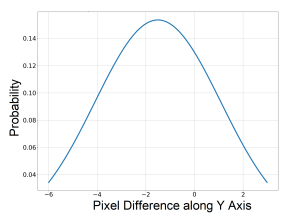
(a) Center x Co-ordinate Deviation of White Piece



(b) Center y Co-ordinate Deviation of White Pieces



(c) Center x Co-ordinate Deviation of Black Pieces



(d) Center y Co-ordinate Deviation of Black Pieces

Fig. 7: Center Pixel Co-ordinate Difference of White and Black Pieces.

three parameters is in continuous domains. Thus, the rule-based algorithm increases the probability of a shot colliding with a piece. Our proposed ruled-based algorithm is having a complexity of, $O(nd)$ where,
n- No.of pieces remaining on the board $(1 \leq n \leq 19)$
d- No.of disk positions (arbitrary constant)
One major challenge in shooting speed (v) calculation is countering the effect of friction of the surface which is not uniform throughout the board. Therefore, we map the calculated speed (pixels per second) from the rule-based algorithm to the manipulator servo input. We present the data and fitted polynomial function in Figure 8. Due to the choice of our design parameters, there is an inverse relationship between the servo input and the shooting level. This mapping counters the effect of friction for more than 90% of the time which is shown in Table IV.

C. Carrom Manipulator

Carrom manipulator plays a central role in the overall system in terms of the gameplay as well as in the training phase. The ability of repeatability with an acceptable accuracy of the system is essential for our implementation. We present

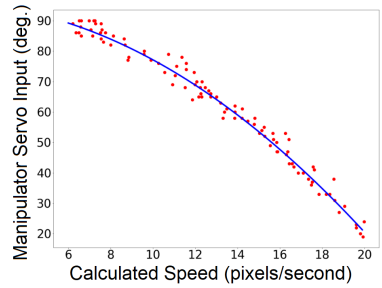


Fig. 8: Manipulator Speed Map

TABLE II: Repeatability of Manipulator: The reliability of the system depends on how well our system can reproduce the same results for the same set of parameters.

Disk Position	Left Top		Right Top		Left Bottom		Right Bottom	
	Angle	Repeatability	Angle	Repeatability	Angle	Repeatability	Angle	Repeatability
1	−11.81	5/5	42.01	4/5	−126.94	5/5	97.41	4/5
2	−16.75	5/5	40.36	5/5	−117.57	5/5	100.4	4/5
3	−21.50	5/5	37.14	5/5	−111.76	5/5	101.64	5/5
4	−25.95	4/5	33.62	3/5	−107.9	5/5	103.19	4/5
5	−30.16	5/5	29.72	4/5	−105.14	5/5	103.26	5/5
6	−33.98	5/5	25.56	3/5	−103.13	4/5	108.02	3/5
7	−37.48	4/5	21.10	4/5	−101.59	4/5	111.93	4/5
8	−40.71	5/5	15.38	4/5	−100.35	4/5	117.9	4/5
9	−43.63	4/5	11.01	5/5	−99.36	4/5	127.61	5/5

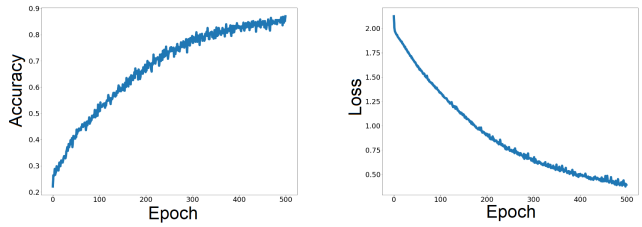
the robustness of our system in the rest of this section. Table II represents the repeatability of our implemented manipulator. It reflects the accuracy of playing the same shot for a given pocket, with a given angle, velocity, and disk position. This experiment is conducted by choosing nine discrete disk positions along the disk placing area. In practice, same set of parameters given to the manipulator may produce distinctive results due to variations of board dynamics over time. Analyzing Table II considering the playtime of a game, we conclude that the implementation is robust, and the accuracy is at an acceptable level for a given gameplay and it is reliable for collecting data. A key factor for this performance is the sensitivity and the range of each mechanism. Table III illustrates the step size, the minimum, and the maximum value that can be attained by each linear, rotational motions and the velocity given to the disk.

TABLE III: Manipulator Specifications

	Resolution	**Min**	**Max**
Linear	0.02mm	−82.56mm	472mm
Rotational	0.3°	−180°	180°
Disk Velocity	0.0289ms^{-1}	0.2910ms^{-1}	2.7459ms^{-1}

D. Reinforcement Learning Algorithm

In literature, the common practice is to use a simulated environment for training algorithms. Here, we discuss the suitability of robotic systems in training reinforcement learning algorithms. In fact, for games like Carrom, this approach is



(a) Disk position accuracy curve (b) Disk position loss curve

Fig. 9: Disk position learning curves

essential due to its stochastic nature which cannot be modeled precisely in dynamic simulations. Training on real-world comes with the inherent inability of repeatability considering the game and its dynamics. Even though, that is the case, overall shot accuracy of 70.6% in Table IV reflects the fact that the robotic system has captured the game dynamics up to a satisfactory level. A deeper network trained on more real data would improve the overall shot accuracy.

Figures 9 and 10 depict the accuracy and loss curves for the two different networks used to predict the disk position, shooting angle and disk velocity. In order to counter the effect caused by the difference in ranges of shooting speeds and shooting angles in the training phase, we scale the inputs such that all speeds and angles are in [0,1] range. In Figure 9 we see non zero gradients in both the accuracy and loss curves which, suggest we can achieve more accuracy with a reduced loss with more data. This reflects the profound capability of our implementation and learning through future improvements.

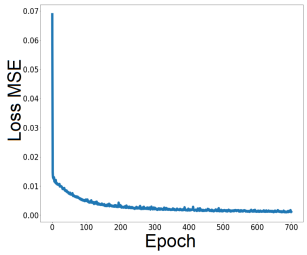


Fig. 10: Accuracy of disk velocity and shooting angle

IV. Discussion

In this paper, we presented a smart robotic system by combining computer vision, heuristic algorithms and robotics to play Carrom. Further, we trained a reinforcement learning algorithm beyond a simulated environment using real data captured using the implemented robotic system. Below, we discuss the challenges we faced for each technical block while discussing the possible future improvements.

In piece detection and localization results, we see some significant accuracy drops in the detection of white pieces compared to black. For different intensity levels, we observe that the difference between the white pieces and the board tends to be very small, making it hard for the detection algorithm to differentiate. Here, we did not consider the option of controlling intensity levels, even though in world championships, light intensity levels are maintained uniformly. Thus, we can obtain good accuracy levels if we control the illumination level. On the other hand, with the development of machine learning and deep learning paradigms, with a properly collected data set, we can implement a more robust version of a detecting algorithm using deep learning techniques as future work.

TABLE IV: Summary of performance/accuracy of the overall system

Computer Vision	Black Pieces Detection Rate	96%
	White Pieces Detection Rate	90%
Rule-Based Speed Mappiing	Manipulator	92%
Manipulator	Repeatability	88%
Reinforcement **Learning Algorithm**	Shooting Speed	MSE: 0.0015
	Shooting Angle	MSE: 0.0012
	Disk Position	Categorical Cross Entropy Loss: 0.3932
		Accuracy 87%
Overall Shot Accuracy	70.6%	

While developing the manipulator, one of the main problems we encountered was determining whether the disk is following the exact path that we intended it to follow. To clarify that, we marked the intended path on the board and compared that with the disk traveling path via a video feed. We intend to automate this procedure using a laser projection associated with a software algorithm as future work.

The method we adopted for training the reinforcement learning algorithms is different from the common practice. If we restricted training to a simulated environment, we expect a significant drop in the efficiency of the manipulator because of the dynamic and stochastic nature of the game. On the other hand, training the learning algorithm on the robotic system is arduous and slow. We leveraged the rule-based algorithm to boost the performance in favor of drastically reducing the exploration time. Another challenge we had to face was the difficulty in recreating a certain state which is the board configuration. Therefore, as future work, we intend to enhance the performance of the system by incorporating a simulated environment and reducing the error between the real-world and the simulation. This approach will give the ability to reproduce any specific state repeatedly as well as to speed up the training process which will eventually lead to a well experienced Carrom playing robot.

V. Conclusion

In this paper, we presented a smart robot that can play Carrom, against a proficient human. We discussed the overall

system implementation as well as the unit level implementations. Further we discussed the challenges and areas to be improved in the future.

The analysis of reinforcement learning results suggests the possibility in adopting a real-world training for a game like Carrom. On the other hand, tackling complex problems such as games like Carrom which has continuous state and action spaces, require intuitive methods to boost the training and learning process. In addition, the idea of adopting heuristic algorithms to reduce the exploration time can be used for many problems in this domain. To the best of our knowledge, our work is the first to attempt a reinforcement learning approach in a robotic system for a cue sport-based tabletop game.

References

[1] M. Greenspan, M. Godard, J. Lam, D. Dupuis, W. Leckie, S. Jordan, I. Zaidi, and K. Anderson, "Toward a competitive pool-playing robot," *Computer*, vol. 41, no. 01, pp. 46–53, Jan 2008.

[2] J. Baxter, A. Tridgell, and L. Weaver, "Reinforcement learning and chess," January 2001, pp. 91–116.

[3] D. Meng, B. Jianbo, Q. Yizhong, F. Yao, and L. Shuqin, "Design of amazon chess game system based on reinforcement learning," in *Chinese Control And Decision Conference*, 2019, pp. 6337–6342.

[4] Y. Zhu, Y. Zhao, L. Jin, J. Wu, and R. Xiong, "Towards high level skill learning: Learn to return table tennis ball using monte-carlo based policy gradient method," in *IEEE International Conference on Real-time Computing and Robotics*, 2018, pp. 34–41.

[5] K. Mülling, A. Boularias, B. Mohler, B. Schölkopf, and J. Peters, "Learning strategies in table tennis using inverse reinforcement learning," *Biological cybernetics*, vol. 108, 2014.

[6] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," 2013.

[7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[8] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[9] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[10] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.

[11] S. Fujimoto, E. Conti, M. Ghavamzadeh, and J. Pineau, "Benchmarking batch deep reinforcement learning algorithms," *ArXiv*, vol. abs/1910.01708, 2019.

[12] H. Wang, T. Zariphopoulou, and X. Zhou, "Exploration versus exploitation in reinforcement learning: A stochastic control approach," *SSRN Electronic Journal*, 01 2019.

[13] H. Van Hasselt, "Reinforcement learning in continuous state and action spaces," 08 2013.

[14] V. R. Konda and J. N. Tsitsiklis, "On actor-critic algorithms," *SIAM J. Control Optim.*, vol. 42, no. 4, p. 1143–1166, Apr. 2003.

[15] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, pp. 9–44, 2005.

[16] T. Vodopivec, S. Samothrakis, and B. Šter, "On monte carlo tree search and reinforcement learning," *J. Artif. Int. Res.*, vol. 60, no. 1, p. 881–936, Sep. 2017.

VI. Appendix

A. System Implementation

In this section, we present the hardware architecture and the calculations related to the manipulator. Figures 12, 13 and 14 show the calculations related to the shooter, the rotator and the linear actuator respectively.

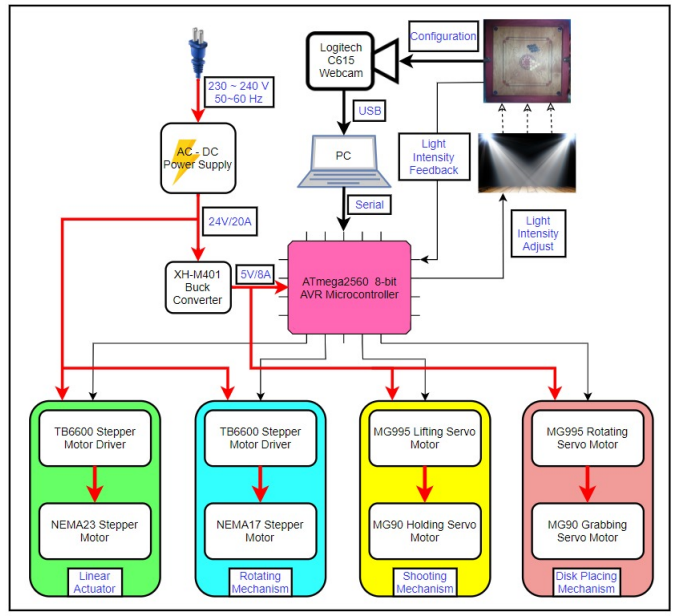


Fig. 11: Hardware Architecture Block Diagram. This diagram shows the system level interconnection of each hardware component along with the power distribution of the end-to-end system.

B. Machine Vision Based Piece Detection and Localization

Algorithm 1 presents the pseudocode used for piece detection and localization. This algorithm is implemented using python language and runs efficiently in real-time.

C. Rule Based Algorithm

Algorithm 2 presents the rule-based algorithm we used. A key feature of employing a rule-based algorithm is that this

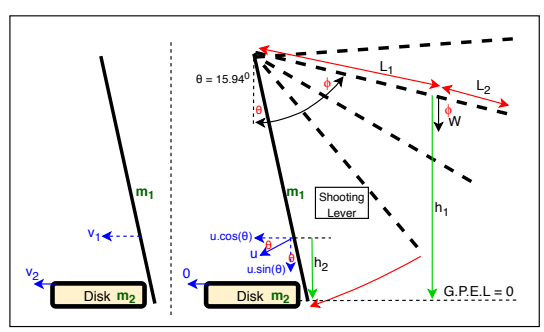


Fig. 12: Calculations of the Shooter: To determine the required weight of the shooting-lever, performed a back calculation starting from the maximum initial speed of the disk.

Algorithm 1: Piece detection and localization algorithm

Input: Image: Image of the carrom board at a given instance
Input: Pocket_templates[Left_top, Left_bottom, Right_bottom, Right_top]
Input: Piece_templates[Black, White, Red]
Input: Thresholds[Pocket(T_h), Black(T_b), White(T_w), Red(T_r)]
Input: Nonmax_threshold(T_i)
Output: Real world coordinates
Initialisation :
coordinateFrame ← 2-D array of size (23,2)
isRedSpotted ← False
Img ← Crop(Image)
for each item in [Img, Pocket_templates, Piece_template] **do**
  Blur_item ← Gaussian_blur(item with kernel_size=5, σ=0)
  Smooth_item ← Blend(Blur_item, item)
  Gray_item ← Gray_scale(Smooth_item)
  Edge_item ← Canny_edge_detection(Gray_item)
end for
for each Edge_template in [Edge_item[!]] **do**
  h, w ← Extract height and width from hole template
  Search through Edge Img_edge $I(x, y)$ sliding Edge template $T(x, y)$
  $R(x,y) = \frac{\sum_{x',y'}(T'(x',y').I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 . \sum_{x',y'} I'(x+x',y+y')^2}}$ Where,
  $T'(x',y') = T(x',y') - \frac{1}{w.h}\sum_{x'',y''} T(x'',y'')$ and
  $I'(x+x',y+y') = I(x+x',y+y') - \frac{1}{w.h}\sum_{x'',y''} I(x+x'',y+y'')$
  Result ← $R(x, y)$
  Locations ← $\{(x, y) \mid$ Result>= T Where T $\in \{T_w, T_r, T_b, T_h\}\}$
  if Edge_template = edge_red_template **then**
    if length(Locations) = 0 $\{$When $T = T_r\}$ **then**
      isRedSpotted ← False
      coordinateFrame ← append([NaN,NaN])
      break
    else
      isRedSpotted ← True
    end if
  end if
  X_L, Y_L ← Locations
  X_L, Y_L ← Nonmax_suppression(X_L, Y_L, Nonmax_threshold, h, w)
  for pt in range(0:length(X_L) **do**
    X = X_L(pt) + int($\frac{w}{2}$)-1
    Y = Y_L(pt) + int($\frac{h}{2}$)-1
    coordinateFrame ← append([X,Y])
  end for
end for
Coordinates ← Sorted(coordinateFrame)
Coordinates ← Convert_to_Homogeneous(Coordinates)
Homogeneous_mat ← H_matrix$_{[3\times3]}$ X Coordinates
Real_world_coordinates ← Homogeneous_mat[First two columns]
return *Real_world_coordinates*

[!] Edge_item = {White_Piece_Edge, Red_Piece_Edge, Black_Piece_Edge, Pocket_Edges}

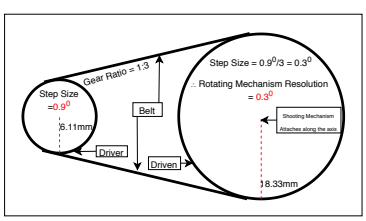


Fig. 13: Calculations of the Rotator: To determine the maximum allowable speed of the motor, first calculated the required torque and matched with motor characteristic graph.

idea can be used for many tabletop games. Also, in reinforcement learning context, this can be used as an advantage to reduce the exploration time.

Algorithm 2: Rule-based algorithm

Input: Location matrix of the pieces and holes (23x2)
Input: Disk location matrix
Output: Final shot
Initialisation :
Piece_to_hole_lines(L_1) ← [ ] (empty array)
Disk_to_piece_lines(L_2) ← [ ]
Black_pieces ← Coordinate matrix[10:19]
White_pieces ← Coordinate matrix[0:9]
Holes ← Coordinate matrix[19:23]
Selected_lines ← [ ]
Possible_shots ← [ ]
δ ← 5
thresh_p ← (piece radius + δ)
thresh_d ← (disk radius + δ)
thresh_a ← 40
Interference_score ← 0
while Possible_shots = $\emptyset$ **do**
  for bp in Black_pieces **do**
    for h in Holes **do**
      L_1 ← append([bp,h,gradient,intercept])
    end for
  end for
  for bp in Black_pieces **do**
    for dl in Disk location matrix **do**
      L_2 ← append([bp,dl,gradient,intercept])
    end for
  end for
  for bp in Black_pieces **do**
    for l_1,l_2 in L_1,L_2 **do**
      S_1 ← [ ] {For Piece - Hole pairs}
      S_2 ← [ ] {For Piece - Disk position pairs}
      S_1 ← count(dist(l_1, Coordinate matrix[:19])<thresh_p)
      min_{S1} ← $_{\text{min}}S_1$
      S_2 ← count(dist(l_2, Coordinate matrix[:19])<thresh_d)
      min_{S2} ← $_{\text{min}}S_2$
      Note: dist is the perpendicular distance from the center of the piece to the piece's travelling path line
      score ← $min_{S1} + min_{S2}$
      if (score = Interference_score) **then**
        Selected_lines ← append([bp,l_1,l_2]—$S_1[l_1]$+$S_2[l_2]$=score)
      end if
    end for
  end for
  for [bp,l_1,l_2] in Selected_lines **do**
    angle ← angle_between_lines(l_1,l_2)
    if (angle<thresh_a) **then**
      Possible_shots ← append([bp,l_1,l_2])
    end if
  end for
  Interference_score + = 1
end while
Select_shot ← random(Possible_shots)
Shot_params ← Shot_parameters(Select_shot)
Shot_params_rl ← RL_shot(Coordinate matrix, Disk location matrix)
Final_shot ← sample(Shot_params, Shot_params_rl, Probability = ϵ)
return *Final_shot*

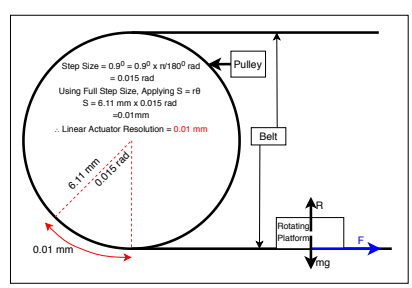


Fig. 14: Calculations of the Linear Actuator: To determine the maximum allowable speed of the motor, first calculated the required torque and matched with motor characteristic graph.